Shared guide: the whole project

For both of you: the story, the pipeline, how to explain each other's half, honest limitations, cross-questions and a glossary.

1. The 20-second pitch

Say this

• We detect 38 plant diseases from leaf photos. We compare a small CNN trained from scratch with a pretrained EfficientNet, and then we test what lab-trained models do on real field photos: we measure the accuracy drop, check with Grad-CAM and a background-swap experiment whether the model looks at the leaf, try stronger augmentation to close the gap, and let the app say 'not sure' when it is unsure.

2. The project story in 2 minutes

Problem. Plant diseases cost farmers a lot of harvest, and experts are not always available. A phone app that recognises a disease from a leaf photo would help, but it has to work on real field photos, not only on clean lab pictures.

Data. We train on PlantVillage: lab photos of single leaves on a plain background, 38 classes, 14 crops. We take a balanced subset (7,430 train / 1,520 val / 2,244 test) and split it by physical leaf to avoid leakage. We test field performance on PlantDoc (2,578 real photos, 28 shared classes), never training on it.

Methods. (1) A small CNN from scratch vs EfficientNet-B0 fine-tuned from ImageNet. (2) Accuracy, macro-F1 and confusion matrices. (3) Grad-CAM heatmaps and a leaf-focus score. (4) A background-bias experiment: training on original, leaf-only and random-background images. (5) Strong augmentation against the lab-to-field gap. (6) A confidence threshold for rejecting unsure predictions. Everything runs in about 40 minutes on a free Colab T4.

Findings. [Read from the tables in the person guides after the Colab run: transfer learning vs scratch on PlantVillage; the size of the domain gap; whether strong augmentation reduced it; whether background swaps hurt the baseline; leaf focus vs leaf area; coverage and accuracy after rejection.]

Limits. Lab-to-field generalisation stays the weak point; augmentation imitates only part of the difference; test sets are small per class; one run per experiment; no field data in training.

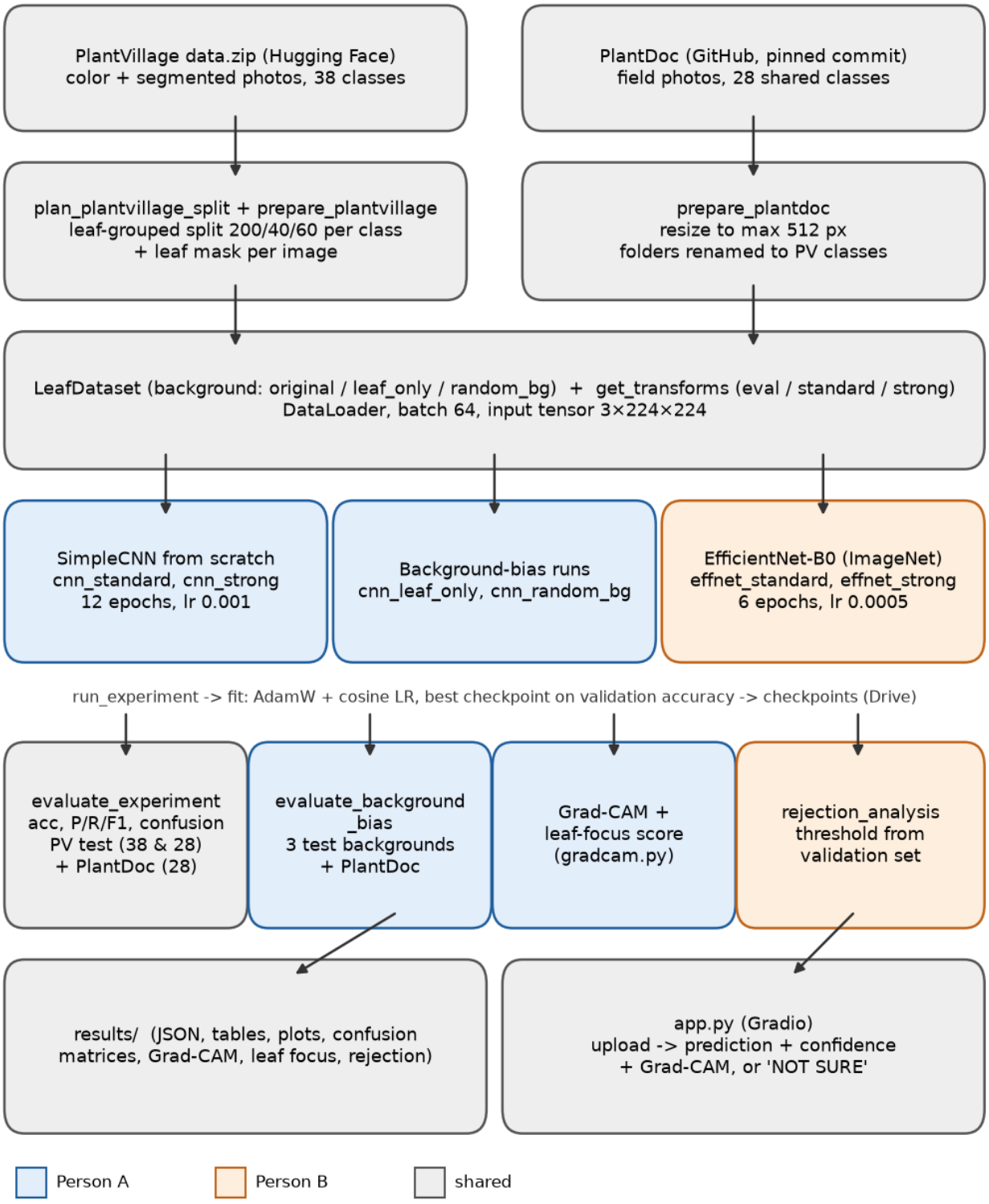
Results are not available yet

• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

Experiment	PV test acc (38)	PV macro-F1	PV acc (28 shared)	PlantDoc acc (28)	PlantDoc macro-F1	Domain gap
cnn_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_strong	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_strong	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_leaf_only	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]

Experiment	PV test acc (38)	PV macro-F1	PV acc (28 shared)	PlantDoc acc (28)	PlantDoc macro-F1	Domain gap
cnn_random_bg	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]

3. The whole pipeline on one page



Blue = Person A, orange = Person B, grey = shared. Function names are the real ones.

4. Explaining the other person's half (5 sentences each)

Person A explains Person B's half

- Person B took EfficientNet-B0, pretrained on ImageNet, replaced its last layer with a 38-class layer, and fine-tuned all layers with a smaller learning rate (0.0005) for 6 epochs.
- Because the network already knows general visual features, it learns faster and generalises better than our scratch CNN.
- B measured the lab-to-field drop on the 28 classes shared with PlantDoc.
- B retrained both models with strong augmentation (random crops, rotation, colour jitter, blur) to imitate field conditions, and compared the gap before and after.
- B added a rejection threshold, chosen on the validation set, so the app says 'not sure' instead of guessing.

Person B explains Person A's half

- Person A designed a small CNN from scratch: 4 conv blocks (32 to 256 filters), global average pooling, dropout and a 38-class output, about 1.18M parameters, trained 12 epochs.
- A evaluated it with accuracy, macro-F1 and a row-normalised confusion matrix, whose diagonal is per-class recall.
- A tested background bias by training on original, leaf-only and random-background images, made with PlantVillage's leaf masks, and testing every model on all three backgrounds.
- A implemented Grad-CAM, which weights the last conv layer's feature maps by the gradients of the class score, to show where the model looks.
- A turned this into a number, the leaf-focus score: the share of the heatmap that lies on the leaf.

5. Shared parts

- **Comparison tables:** `results_table` (evaluate.py) writes `results/results_table.md`; `background_table` writes `results/background_table.md`; the notebook writes `results/leaf_focus_table.md`.
- **Gradio demo:** `app.py` (local: `python app.py`; in Colab: last notebook cell with `--share`).
- **Colab runs:** the notebook `notebooks/train_plant_disease.ipynb` runs everything in order. Its sections: 0. Settings; 1. Setup: GPU, code, packages, Google Drive; 2. Data; 3. Baseline: small CNN trained from scratch; 4. Transfer learning: EfficientNet-B0 pretrained on ImageNet; 5. Evaluation on the PlantVillage test set; 6. Domain shift: lab photos → field photos; 7. Reducing the gap with stronger augmentation; 8. Background-bias experiment: does the model look at the leaf or at the background?; 9. Grad-CAM: where does the model look? + leaf-focus score; 10. Confidence threshold: say "not sure" instead of guessing; 11. Save results to Google Drive and download them; 12. (Optional) Gradio demo inside Colab.
- **README:** overview, how to run, results table (fill in after the run), cheat sheet.
- **These PDFs:** `python scripts/dataset_stats.py` then `python scripts/build_viva_pdfs.py`.

6. Honest limitations, and how to say them confidently

Limitation	How to say it
PlantVillage is a lab dataset with plain backgrounds	"We knew lab accuracy would be optimistic, so we measured the field gap directly instead of hiding it."
Small test sets (≤60 per class; PlantDoc classes small)	"Per-class numbers are noisy; we rely on overall accuracy and macro-F1 and do not over-interpret single classes."
Only one run per experiment	"Differences of 1-2 points may be noise; with more compute we would repeat with several seeds."
Only 28 of 38 classes exist in PlantDoc	"We compare on exactly the shared classes, so the comparison is fair."
No field images in training	"That was on purpose, to measure the gap; the obvious next step is to add some."
Grad-CAM and leaf focus are approximate	"We use them as evidence, not proof, and cross-check with the background experiment."
Confidence is not calibrated	"Rejection reduces wrong answers but cannot remove them; calibration would be future work."
We did not try: other architectures, hyper-parameter search, feature extraction, domain adaptation	"We focused on a clean comparison within a free-GPU budget."

7. 20 cross-questions (either of you)

Q1. Plant disease classification on PlantVillage has been done many times. What is new in your project?

We do not claim a new model. Our contribution is an honest evaluation: lab vs field accuracy on the same shared classes, a background-bias test with leaf masks, a measurable Grad-CAM leaf-focus score, and a rejection threshold, all reproducible on free hardware.

Q2. Why is accuracy not enough?

It hides per-class problems (a small class can be always wrong while total accuracy stays high), it says nothing about why the model decides (shortcuts), and lab accuracy says little about field performance. So we add macro-F1, confusion matrices, Grad-CAM/leaf focus and the PlantDoc test.

Q3. How did you prevent data leakage?

Train/val are split by physical leaf using the leaf map; test is the authors' official leaf-grouped split; PlantDoc is never used for training; the rejection threshold and best epoch are chosen on validation only. Residual: 2 leaf groups are shared between train and test through the official split.

Q4. What is domain shift, and how big is it here?

A difference between the training and deployment data distributions; here lab vs field photos. The size is the 'domain gap' column in our table: PV accuracy minus PlantDoc accuracy on the same shared classes. [quote numbers]

Q5. Why did you use PlantDoc only for testing?

To measure how a lab-trained model behaves on photos it has never seen, which is the realistic situation. Training on it would remove our only out-of-domain test.

Q6. Would you deploy this model for farmers?

Not as it is. Field accuracy is lower than lab accuracy, confidence can be wrong, and it only knows 38 classes. It could be an assistant that says 'not sure' and points to an expert, after training on real field data and testing in the target region.

Q7. What are the ethical risks?

A wrong diagnosis could lead to wrong or unnecessary pesticide use, or to missing a disease and losing a crop. Users might over-trust a confident app. The data come from a few crops and regions, so the model may be biased. Hence rejection, clear disclaimers and expert follow-up.

Q8. Why is the CNN trained from scratch at all if transfer learning is better?

It is the baseline. Without it we could not show how much pretraining helps; it is also the model in the background-bias experiment, because a scratch model can only learn from our data.

Q9. How reproducible are your results?

Seeds are fixed (set_seed, seed_worker, seeded generator), cudnn is deterministic, the PlantDoc commit is pinned, and the split comes from deterministic code. Small GPU differences are still possible, and each experiment ran once, so we report no variance.

Q10. Explain Grad-CAM in one sentence.

The gradient of the class score tells how important each feature map of the last conv layer is; the weighted sum of the maps, after ReLU and upsampling, shows where the evidence for that class is.

Q11. Is Grad-CAM a proof of what the model does?

No, it is an approximation at low resolution. That is why we add the leaf-focus number and the background-swap experiment, which test the same question in different ways.

Q12. Your test set is small. Is that a problem?

At most 60 images per class (2,244 in total; some classes fewer), so per-class numbers can vary by several points. Overall accuracy over 2,244 images is fairly stable. PlantDoc classes are also small.

Q13. Why these hyper-parameters? Did you tune them?

No systematic tuning; they are standard values (Adam-type optimizer, 1e-3 scratch / 5e-4 fine-tuning, cosine schedule) chosen to fit the 40-minute free-GPU budget. Validation accuracy picks the epoch.

Q14. What does the 'other' column in the PlantDoc confusion matrix mean?

Predictions of a PlantVillage class that PlantDoc does not have (10 such classes). They are always wrong, and show that the model is confused by field images.

Q15. Why do you resize to 224×224?

That is the resolution EfficientNet-B0 was trained on with ImageNet; we use the same input for the CNN for a fair comparison. It is also fast.

Q16. What would you do differently with more time and compute?

Train on some labelled field images, try more models and seeds, tune hyper-parameters, calibrate confidence, use the full dataset, and test on photos from our own region.

Q17. What is the difference between validation and test sets in your project?

Validation is used during development (best epoch, rejection threshold). The test sets (PlantVillage test, PlantDoc) are only used once at the end for the reported numbers.

Q18. How does the Gradio app work?

app.py loads a checkpoint, resizes and normalises the uploaded photo, runs the model and Grad-CAM, shows the top-5 probabilities and the overlay, and says 'NOT SURE' when the confidence is below the saved threshold.

Q19. Why does the model sometimes predict a class from the wrong crop on field photos?

Field photos contain clutter and other textures, and the model never learned to ignore them. It also never learned 'which crop is this' separately; it picks among all 38 classes at once.

Q20. Who did what?

Person A: baseline CNN, its evaluation, the background-bias experiment, Grad-CAM and leaf focus. Person B: transfer learning, its evaluation, the augmentation experiments, the rejection threshold. Together: the comparison tables, the Gradio demo, the Colab runs, the README and the report.

8. Glossary

Term	Meaning in one sentence
Accuracy	Share of all test images that were classified correctly.
Activation / feature map	The output grid of a conv layer; large values where its pattern was found.
AdamW	Optimizer that adapts the step size per weight and applies weight decay separately.
Augmentation	Random label-preserving changes to training images (flip, crop, colour, blur) to add variety.
Backpropagation	Computing the gradient of the loss for every weight, layer by layer from output to input.
Background bias / shortcut	When a model uses an unrelated clue (like the background) instead of the real evidence.
Batch	A group of images processed together; ours has 64 images, shape 64×3×224×224.
BatchNorm	Normalises each channel over the batch, then rescales it; makes training stable.
Checkpoint	A saved copy of the model weights (.pt file), here the epoch with the best validation accuracy.
Class imbalance	Some classes have many more images than others.
Confidence	The softmax probability of the predicted class.
Confusion matrix	Table of true class (rows) vs predicted class (columns); ours is row-normalised.
Convolution	Sliding a small learned filter over the image and summing the products at each position.
Cosine annealing	Learning-rate schedule that decreases the LR along half a cosine curve.
Coverage	Share of images the system answers instead of rejecting.
Cross-entropy loss	Loss that is small when the model gives a high probability to the true class.
Data leakage	Information from the test data getting into training, making results look too good.
Domain gap	PlantVillage accuracy minus PlantDoc accuracy on the same shared classes.
Domain shift	Test/deployment data differ in distribution from the training data.
Dropout	Randomly zeroes some features during training so the model cannot rely on any single one.
EfficientNet-B0	A small, efficient CNN made of MBConv blocks; our pretrained model.
Epoch	One pass over all training images.
F1 score	Harmonic mean of precision and recall for one class.
Feature extraction	Transfer learning with the pretrained layers frozen; only the new head is trained.
Fine-tuning	Transfer learning where the pretrained layers are also updated (what we did).
Global average pooling	Averages each feature map to one number.
Grad-CAM	Heatmap of the image regions that increased the score of a class, from last-layer gradients.
Gradient	How much the loss (or a score) changes when a weight or activation changes a little.
ImageNet	1.28M labelled photos in 1000 classes, used to pretrain our EfficientNet.
Leaf-focus score	Share of the Grad-CAM heat that lies inside the leaf mask.

Term	Meaning in one sentence
Leaf mask	Black-and-white image: white = leaf pixel, made from PlantVillage's segmented photo.
Learning rate	Size of each weight update step.
Logit	Raw class score before softmax.
Macro average	Average of a per-class metric where every class counts equally.
Max pooling	Keeps the largest value in each small window, shrinking the feature map.
Mixed precision	Using 16-bit floats on the GPU for speed, with a GradScaler for safety.
Normalisation (input)	Subtracting the mean and dividing by the std per colour channel.
Overfitting	Model memorises training data and does worse on new data.
PlantDoc	Dataset of real field photos of leaves; our out-of-domain test set.
PlantVillage	Dataset of lab photos of single leaves on a plain background; our training data.
Precision	Of the images predicted as class c, the share that really are c.
Recall	Of the images that really are class c, the share the model found.
Rejection threshold	Minimum confidence needed to give an answer; below it the app says 'not sure'.
ReLU	Activation function max(0, x).
Seed	Fixed starting value for random number generators so runs repeat exactly.
Softmax	Turns logits into probabilities that sum to 1.
Tensor	A multi-dimensional array; an image is 3×224×224.
Transfer learning	Starting from a model trained on another task instead of random weights.
Validation set	Data used during development to pick the best epoch and the threshold, never for the final score.
Weight decay	Penalty that keeps weights small, to reduce overfitting.

Things to double-check before the viva

These are places where the code is easy to misdescribe, or where something is limited or undocumented. Better to know them than to be surprised.

- The files now in `results/` come from a 1-epoch CPU smoke test on 2 images per class. They are NOT results. This builder detects that and prints [pending]. Run the Colab notebook, copy `results/` back, then run `python scripts/build_viva_pdfs.py` again.
- README.md says "see 'Getting it onto Colab' below", but that section does not exist in the README.
- Training on the command line (`python -m src.train`) uses batch size 32 and lr 1e-3 for BOTH models by default. The notebook uses batch 64, lr 0.001 (CNN) and 0.0005 (EfficientNet). Always quote the notebook values; those produced the results.
- `fit` (train.py) always runs ALL epochs and only keeps the checkpoint with the best validation accuracy. The code comment says "early-stopping style"; it is not real early stopping (training never stops early).
- `build_efficientnet(freeze_backbone=True)` exists, but `get_model` never passes it, so feature extraction was never run. We have no numbers for it; only describe it as an option.
- The eval transform is `Resize((224, 224))`, which squashes non-square photos. PlantVillage photos are square (256×256), but PlantDoc photos are not (e.g. 512×384), so they get distorted. This is part of the measured domain gap.
- Leakage: our train and val share 0 leaf groups, but train and test share 2 leaf groups. These come from the authors' official split, which we use as is. Also, photos missing from the leaf map count as their own group, so the check cannot be 100% complete.

- Leaf masks: pixel = leaf if its brightest channel is > 25 in the segmented image, then a 5×5 median filter. The largest leaf share among our local masks is 1.00 (1.00 = the mask covers the whole image), so some masks are poor. Dark lesions or shadows can be misclassified.
- In `leaf_only` the background is solid black, which is itself one constant background. `random_bg` backgrounds are synthetic colour blobs, not real field scenes. Say this when you present the experiment.
- Leaf-focus score: the CAM is low-resolution (SimpleCNN 14×14, EfficientNet 7×7) and is blown up to 224×224, so heat spills over the leaf border. The score cannot tell lesion from healthy leaf, and it can only be computed on PlantVillage (PlantDoc has no masks).
- Grad-CAM's target layer for SimpleCNN is `features[-1]`, the whole 4th conv_block, including its MaxPool (output 256×14×14). Call it 'the output of the last conv block'.
- The rejection threshold is chosen on the in-domain (lab) validation set. Softmax confidence is not calibrated, so field photos can be confidently wrong. The threshold is computed only for the two EfficientNet models. app.py falls back to 0.5 if `results/rejection/<name>.json` is missing.
- `RandomRotation(30)` fills the corners with black, and `RandomResizedCrop(scale 0.5-1)` can crop away the diseased spot, so some augmented images may no longer show the disease.
- PlantDoc's spider-mites folder has only 2 images, and several PlantDoc classes are small, so per-class field numbers are noisy. The PlantVillage test set has at most 60 images per class (Potato healthy 32, Apple cedar rust 52).
- Each experiment is run once with seed 42. Differences of 1-2 points between models may be noise. We report no confidence intervals or repeated runs.
- How PlantVillage was collected comes from the dataset paper (Hughes & Salathé 2015; Mohanty et al. 2016), not from our code. Check the wording against the paper before quoting details.
- The cosine LR schedule is stepped once per epoch with T_max = number of epochs and no warm-up, so the last EfficientNet epoch uses lr 3.35e-05 (see the LR table).
- The "~40 min on a T4" estimate is not measured yet. Read `train_time_sec` in `results/history_*.json` after the run.